

Control de Periféricos Digitales Integración de Sistemas Embebidos

UCC – Software de Sistemas Embebidos - 2026

Dr. Ing. Agustín Laprovitta (alaprovitta@ucc.edu.ar)
Ing. Martín Molina (2369147@ucc.edu.ar)

TRABAJO PRÁCTICO N°1

Objetivos de Aprendizaje

Al finalizar este trabajo práctico, el alumno será capaz de:

1. Configurar y gestionar entradas/salidas digitales (GPIO) en microcontroladores STM32F103 mediante la librería HAL.
2. Diferenciar entre temporización bloqueante (HAL_Delay) y no bloqueante (HAL_GetTick), comprendiendo el impacto en el flujo de ejecución del programa.
3. Controlar registros de desplazamiento (74HC595) mediante señal de reloj, datos y latch (bit-banging) para el manejo de displays de 7 segmentos.
4. Implementar técnicas de lectura y filtrado (antirrebote/debounce) en pulsadores de manera no bloqueante.
5. Integrar múltiples tareas independientes en una máquina de estados o bucle no bloqueante, aplicando conceptos de desarrollo de firmware concurrente y debugging.

Consignas del Trabajo Práctico

Modulo 1: Manejo de LEDs y Temporizaciones

Objetivo: Manipulación de pines GPIO y control del tiempo.

1. Paso A (Delay Bloqueante): Configure los 4 LEDs del shield HW-262. Programe un parpadeo simultáneo cada 500 ms de los 4 LEDs utilizando la función bloqueante HAL_Delay().
2. Paso B (Temporización No Bloqueante): Modifique el código anterior para utilizar HAL_GetTick(). Los LEDs deben conmutar sin detener la ejecución del bucle principal (while(1)) según la siguiente secuencia: D1 cada 250 ms, D2 cada 500 ms, D3 cada 750 ms y D4 cada 1 s.
3. Paso C (Efectos de Luces): Implemente dos funciones separadas:
 - Efecto Auto Fantástico: Un LED encendido se desplaza secuencialmente de izquierda a derecha y viceversa (D1 -> D2 -> D3 -> D4 -> D3 -> D2) con un intervalo de actualización fijo de 500 ms.
 - Contador Binario: Muestre el conteo de 0 a 15 (0000₂ a 1111₂) en los 4 LEDs con un intervalo de actualización fijo de 1 s.

Modulo 2: Multiplexado de Displays 7 Segmentos con 74HC595

Objetivo: Comunicación con registros de desplazamiento y renderizado numérico.

1. Identifique los pines asignados al registro de desplazamiento 74HC595 en el Shield HW-262 (Latch, Clock, Data).
2. Desarrolle la rutina de envío de datos serie (Bit-Banging) para escribir el código del dígito a mostrar en el registro de desplazamiento.
3. Cree un contador decimal que se incremente automáticamente cada 1 segundo sin bloquear la ejecución. Muestre el valor en los displays de 7 segmentos.

Modulo 3: Lectura de Pulsadores y Antirrebote

Objetivo: Procesamiento de eventos externos y filtrado de ruido mecánico.

1. Configure los 3 pulsadores integrados (S1, S2, S3) en modo entrada digital (`GPIO_Input` con resistencia pull-up interna si aplica según el esquema del HW-262).
2. Implemente una función de lectura con algoritmo de antirrebote por software (por tiempo mediante `HAL_GetTick()`).
3. Compruebe la lectura encendiendo o apagando un LED individual únicamente mientras el pulsador esté presionado.

Modulo 4: Integración del Sistema y Debugging (Desafío Final)

Objetivo: Integración de periféricos en una arquitectura concurrente.

Escriba un único programa que ejecute simultáneamente las siguientes funcionalidades sin que una bloquee a las demás:

- Reloj principal: El display de 7 segmentos muestra un contador ascendente que se incrementa cada 1 segundo.
- Acción Pulsador S1: Al presionar S1, el contador del display se reinicia inmediatamente a 0.
- Acción Pulsador S2: Activa el efecto de Auto Fantástico en los 4 LEDs.
- Acción Pulsador S3: Activa el modo de Contador Binario en los 4 LEDs.
- Alarma sonora: El buzzer emitirá un pitido (beep) de corta duración (100 ms) de forma automática cada 5 segundos, sin interferir con el avance del contador ni los efectos visuales.



Nota de Debugging: Utilice el depurador (Hardware Debugger de STM32CubeIDE) con puntos de interrupción (breakpoints) y ventana de variables Live Expressions para verificar el estado interno de las variables del contador y la selección del efecto activo en los LEDs.

Consigna para la Entrega de Informe (Aula Virtual)

Formato y Condiciones de Entrega

- Formato de archivo: PDF (TP_GPIO_Apellido_Nombre.pdf).
- Extensión máxima: 3 páginas (formato Ficha Técnica o *Reporte Ejecutivo*).
- Plazo de entrega: 21/10/2026.
- Modalidad: Individual.

Estructura y Contenido del Informe

1. Encabezado

- Nombre completo del estudiante.
- Firma y fecha de realización.

2. Mapeo de Hardware y Configuración de Pines

Complete una tabla resumen indicando la configuración aplicada en STM32CubeMX:

Periférico	Pin STM32	Modo GPIO (Output, Input, etc.)	Configuración de Pull (Up, Down, NoPull)	Lógica de Activación (High / Low)

Diagrama de Flujo o Esquema del Algoritmo

- Incluya un diagrama simple (de bloques o de estados) que explique cómo el programa principal gestiona concurrentemente los temporizadores (HAL_GetTick), la lectura de botones y la actualización del display sin usar retardos bloqueantes.

Evidencia de Funcionamiento

- Adjunte un Video corto (link de Drive/YouTube) que muestre el sistema operando:
 - El contador incrementando en el display.
 - El cambio de efecto visual en los LEDs al presionar los pulsadores S1, S2 o S3.

Análisis Técnico y Conclusión Breve (Responda en 3 o 4 líneas)

1. ¿Qué complicación física o lógica enfrentó durante la integración y cómo la resolvió usando el depurador (*debugger*)?
2. ¿Por qué fue indispensable el uso de HAL_GetTick() en lugar de HAL_Delay() para cumplir con la consigna de la alarma de 5 segundos?
3. Comentarios y conclusiones del trabajo realizado.

Criterios de Evaluación

Criterio	Sobresaliente (10 - 9)	Satisfactorio (8 - 7)	En Proceso (6 - 4)	Incompleto (3 - 1)
1. Configuración de GPIO y Temporizaciones	Configura correctamente los pines. Implementa HAL_GetTick() sin bloqueos de forma limpia.	Configura los pines pero requiere correcciones menores en la lógica no bloqueante.	Depende del uso de HAL_Delay() bloqueando el sistema.	Mala configuración de pines GPIO; no funciona.
2. Manejo de Displays y 74HC595	Controla el registro de desplazamiento eficientemente; la cuenta incrementa con precisión de 1 s.	Muestra la cuenta pero con ligeros parpadeos o errores de formato numérico.	Muestra datos erróneos en el display o usa retardos bloqueantes.	No logra comunicarse con el shift register.
3. Pulsadores y Antirrebote	Detecta pulsaciones de forma precisa sin lecturas falsas usando antirrebote por software.	Detecta las entradas correctamente pero presenta ocasionales rebotes/falsos disparos.	Funciona intermitentemente o no implementa técnica de antirrebote.	No lee las entradas de los pulsadores.
4. Integración y Concurrencia (Código Final)	Todas las funciones (display, LEDs, pulsadores, buzzer) operan de manera simultánea e independiente.	Integra las funciones pero el buzzer o los pulsadores generan pequeñas interrupciones / retrasos.	La integración funciona parcialmente; una función bloquea la ejecución de otra.	No logra integrar las partes en un solo código.
5. Debugging y Calidad de Código	Código modular, documentado y uso adecuado del depurador para justificar fallos.	Código legible; utiliza el depurador con ayuda del docente.	Código desorganizado; escaso uso de funciones o variables globales excesivas.	Código no ejecutable o copiado sin comprensión.